Mockito Verification

Junit Assertion:

- It focus on return values.
- Does not care how it happened.
- Only validates whether system produced the expected value or not.

```
int sum = calculator.add(5, 5);
assertEquals(10, sum);
```

Mockito Verify:

- It focus on **BEHAVIOR**.
- It does care the internal method calls.
- How different methods collaborated, which method called how many times and in what sequence, with what parameters etc.
- So in simple term, it validates if internal flow of the method is correct or not.

Before we see an example of each different ways of verification, below is a quick one liner cheat sheet of Mockito verification types:

VERIFICATION CHEATSHEET:

BASIC		
	verify(mock).method()	Called exactly once (default)
	verify(mock, times(n)).method()	Called exactly n times
	verify(mock, never()).method()	Never called
COUNT		
	verify(mock, atLeastOnce()).method()	Called ≥ 1 time

	verify(mock, atLeast(n)).method()	Called ≥ n times
	verify(mock, atMostOnce()).method()	Called ≤ 1 time
	verify(mock, atMost(n)).method()	Called ≤ n times
	verify(mock, only()).method()	Only this method called on mock
ARGUMENT MATCHING		
	verify(mock).method(any())	Any argument
	verify(mock).method(eq(value))	Exact value match
	verify(mock).method(isNull())	Null argument
	verify(mock).method(argThat(...))	Custom condition
	ArgumentCaptor<T> captor verify(mock).method(captor.capture())	Capture argument for inspection
ORDER		
	InOrder	Verifies that interactions happen in a specific sequence across mocks
NO INTERACTIONS		
	verifyNoInteractions(mock)	Mock never touched
	verifyNoMoreInteractions(mock)	No unexpected extra calls
ASYNC		
	verify(mock, timeout(ms)).method()	Called within given time

	verify(mock, after(ms)).method()	Verify after delay

1. **verify(mock).method()**

UserServiceTest.java

```java
@Test
void testSimpleVerify() {
    // Arrange
    User user = new User(1L, "John");
    UserRepository repository = mock(UserRepository.class);
    doNothing().when(repository).save(user);
    UserService service = new UserService(repository);

    // Act
    service.register(user);

    // Assert - verify if save() method got called with this user
    verify(repository).save(user);
}
```

UserService.java

```java
public class UserService {

    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }

    public void register(User user) {
        repository.save(user);
    }
}
```

This is equals to :
verify(repository, times(1)).save(user);

Which means, when we did "*service.register(user)*" , within that flow, ***repository.save(user)*** method should be invoked only once.

Pls Note: Verify accepts only Mock or Spy.

You might ask why?
If we recall from Mockito Architecture video, I showed the flow, in which when we work with Mock, the each method invocation is intercepted and recorded in ***InvocationContainer***

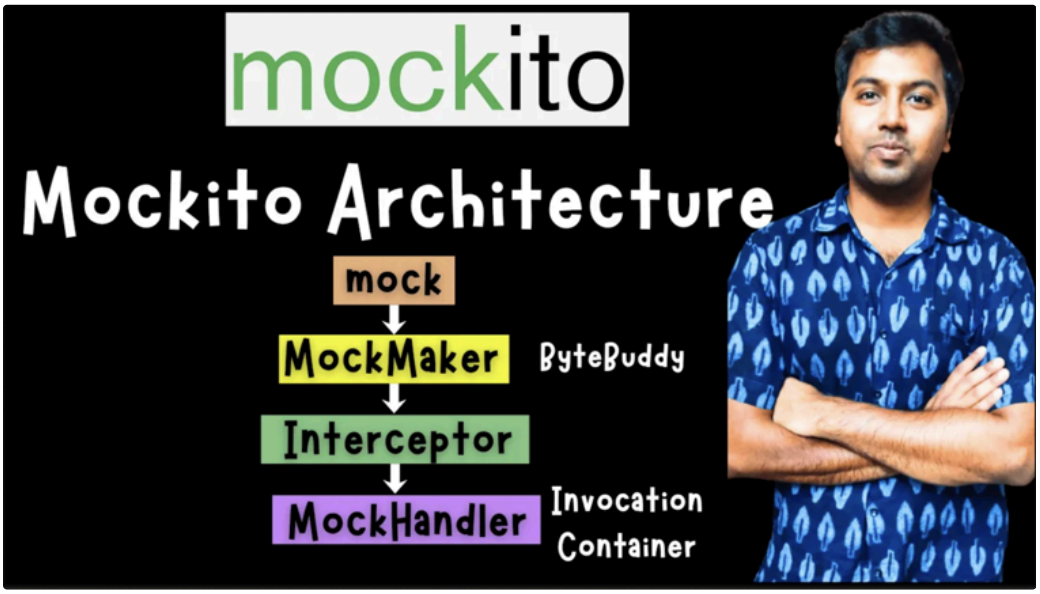
This container stores details like:

- which method was called
- with what arguments
- how many times it was invoked

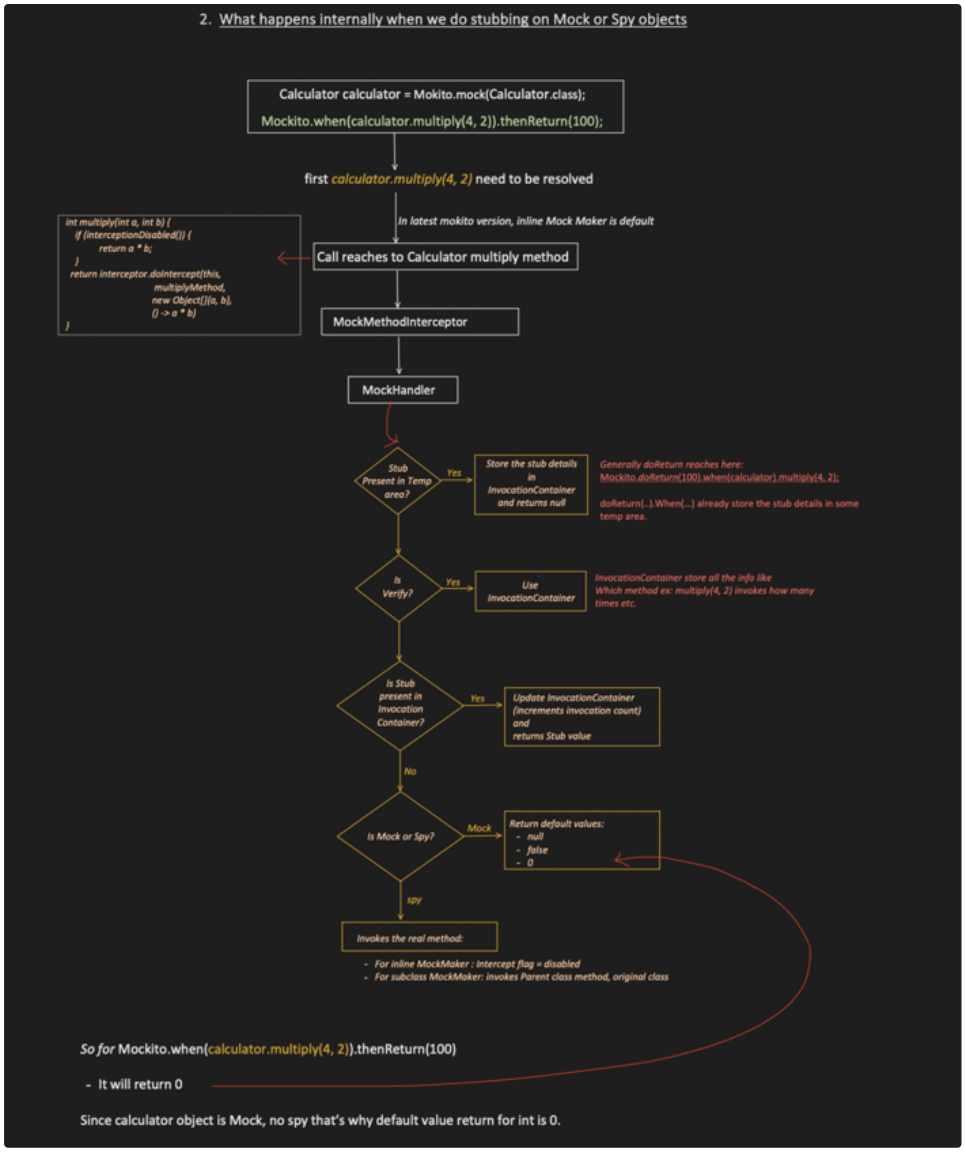
Later, when we call ***verify(...)***, Mockito checks this InvocationContainer to validate the interaction.

For real objects:

No interception happens → No recording → No verification possible



2. **What happens internally when we do stubbing on Mock or Spy objects**



2. **times(n) and never()**

```java
@Test
void testTimesExact() {
    Calculator calc = mock(Calculator.class);
    calc.add(1, 2);
    calc.add(1, 2);
    calc.add(1, 2);

    verify(calc, times(3)).add(1, 2); //Passes - exactly 3 times

verify(calc, never()).multiply(1, 2); //Passes - multiply method with parameter (1,2) never invoked
}
```

3. **atLeastOnce(), *atLeast*(n), *atMostOnce(), atMost(n), only()*:**

```java
/**
```

```
 * atLeastOnce() - At minimum ONE call
 */
@Test
void testAtLeastOnce() {
    Calculator calc = mock(Calculator.class);
    calc.add(1, 2);
    calc.add(3, 4);
    calc.add(5, 6);

    verify(calc, atLeastOnce()).add(anyInt(), anyInt());  // 3 >= 1
}
```

```
/**
 * atMostOnce() - Maximum ONE call (0 or 1)
 */
@Test
void testAtMostOnce() {
    Calculator calc = mock(Calculator.class);
    calc.add(1, 2);

    verify(calc, atMostOnce()).add(anyInt(), anyInt());  // 1 <= 1
}
```

```
/**
 * only() - Called ONCE and NO OTHER methods on mock
 * were called
 */
@Test
void testOnly() {
    Calculator calc = mock(Calculator.class);
    calc.add(1, 2);

    verify(calc, only()).add(1, 2);  // ONLY add() called, nothing
else
}
```

```
/**
 * atLeast(n) - At minimum N calls
 */
@Test
void testAtLeastN() {
    Calculator calc = mock(Calculator.class);
    calc.add(1, 2);
    calc.add(3, 4);
    calc.add(5, 6);

    verify(calc, atLeast(2)).add(anyInt(), anyInt());  // 3 >= 2
}
```

```
/**
 * atMost(n) - Maximum N calls
 */
@Test
void testAtMostN() {
    Calculator calc = mock(Calculator.class);
    calc.add(1, 2);
    calc.add(3, 4);

    verify(calc, atMost(5)).add(anyInt(), anyInt());  // 2 <= 5
}
```

4. **Argument Matching:**

   Kindly check Mockito Stubbing topic, in that I have discussed about **Matchers**:

anyInt():

We don't care about exact values.

```
@Test
void testAnyMatcher() {
    Calculator calc = mock(Calculator.class);

    calc.add(5, 10);

    verify(calc).add(anyInt(), anyInt());
}
```

eq(...) :

We want strict validation of arguments

```
@Test
void testEqMatcher() {
    Calculator calc = mock(Calculator.class);

    calc.add(5, 10);

    verify(calc).add(eq(5), eq(10));  // Must match exactly 5 and 10
}

```

intThat(...)/argThat(...) → custom condition or parameter:

Generally used, when we want to add validation on parameter.

argThat = use when argument is Object, because this execute lambda expression and return null.

intThat = use when argument is int, because this execute lambda expression and return 0.
similarly we have:

- longThat(...)
- floatThat(...)

```
@Test
void testIntThatMatcher() {

  Calculator calc = mock(Calculator.class);

    calc.add(5, 10);

    verify(calc).add(
                    intThat(a -> a > 3), // first param condition
                    eq(10) // second param, we can use argThat(…) or any matcher
                    );
}

```

Argument Capture:

With argThat(...) we can add validation for each parameter, but what if we need to add cross parameter validation.

```
@Test
void testArgThatMatcher() {

  Calculator calc = mock(Calculator.class);

    calc.add(5, 10);
      /*
      Not supported
      */
    verify(calc).add(argThat((a, b) -> a + b > 50));
}

```

```
ArgumentCaptor<Integer> aCaptor = ArgumentCaptor.forClass(Integer.class);
ArgumentCaptor<Integer> bCaptor = ArgumentCaptor.forClass(Integer.class);

verify(calc).add(aCaptor.capture(), bCaptor.capture()); //Captor is also a matcher, if 1 is use Captor, all other should be matcher only.
      /*
        Will return Integer.

        If our captor is like:
        ArgumentCaptor<User> uCaptor = ArgumentCaptor.forClass(User.class);
        User user = uCaptor.getValue(); // will return User.
      */
assertTrue(aCaptor.getValue() + bCaptor.getValue() > 50);
```

```
13
```

5. **Order :**

- This is required to ensure dependencies are invoked in correct sequence.

```java
@Test
void testInOrderSingleMock() {
    Calculator calc = mock(Calculator.class);

    calc.add(1, 2);
    calc.multiply(3, 4);
    calc.add(5, 6);

    InOrder inOrder = Mockito.inOrder(calc);

    // Verify in sequence
    inOrder.verify(calc).add(1, 2);       // First
    inOrder.verify(calc).multiply(3, 4); // Second
    inOrder.verify(calc).add(5, 6);       // Third
}
```

```java
@Test
void testInOrderMultipleMocks() {
    PaymentGateway gateway = mock(PaymentGateway.class);
    OrderRepository orderRepo = mock(OrderRepository.class);
    NotificationService notifier = mock(NotificationService.class);

    // workflow: charge → save → notify
    gateway.charge(100);
    orderRepo.save();
    notifier.sendMail("order@test.com");

    // Verify order across ALL mocks
    InOrder inOrder = Mockito.inOrder(gateway, orderRepo, notifier);

    inOrder.verify(gateway).charge(100);  // First customer should be
charged
    inOrder.verify(orderRepo).save(); // second order need to be saved
    inOrder.verify(notifier).sendMail("order@test.com"); // third mail
need to be send
}
```

6. **verifyNoInteractions(...) and verifyNoMoreInteractions(...):**

```java
/**
 * verifyNoInteractions - Mock was NEVER touched at all
 */
@Test
void testVerifyNoInteractions() {
    UserRepository repository = mock(UserRepository.class);
    NotificationService notifier = mock(NotificationService.class);

    // Only repository is used
    repository.findById(1L);

    // Verify notifier was never touched
```

```
    verifyNoInteractions(notifier);

  }

```

```
/**
 * verifyNoMoreInteractions - After verified calls, nothing else
 */
@Test
void testVerifyNoMoreInteractions() {
    Calculator calc = mock(Calculator.class);

    calc.add(1, 2);
    calc.multiply(3, 4);

    // Verify expected calls
    verify(calc).add(1, 2);
    verify(calc).multiply(3, 4);

    // Ensure NOTHING ELSE was called
    verifyNoMoreInteractions(calc);
}

```

7. **timeout(...) and after(...):**

```
public class AsyncOrderService {

    private final OrderRepository orderRepository;
    private final NotificationService notificationService;
    private final ExecutorService executor =
Executors.newSingleThreadExecutor();

    public AsyncOrderService(OrderRepository orderRepository,
NotificationService notificationService) {
        this.orderRepository = orderRepository;
        this.notificationService = notificationService;
    }

    public boolean placeOrder(String customerEmail) {
        // Step 1: Save order immediately
      /*
      Save the order
        */
        orderRepository.save();

        // Step 2: Send email in BACKGROUND (non-blocking)
        /*
        In Async, send mail
        */
        executor.submit(() -> {
            // Simulate network delay for email
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {

            }
            notificationService.sendMail(customerEmail);
        });

        // Step 3: Return immediately (don't wait for email)
        return true;
    }
```

```
}

```

AsyncOrderServiceTest.java

```
@Test
void testAsyncOrderService_WithTimeout() {
    // Arrange
    OrderRepository orderRepo = mock(OrderRepository.class);
    NotificationService emailService =
mock(NotificationService.class);
    AsyncOrderService service = new AsyncOrderService(orderRepo,
emailService);

    // Act
    service.placeOrder("sj@xyz.com");

    // Assert 1: Order saved immediately (sync)
    verify(orderRepo).save();

    // Assert 2: since sendMail(…) runs in Async, it may execute after
placeOrder(..) method returns. But our verify runs immediately.
    //so its possible that our verify(..) runs before the Async thread
invokes "sendMail(…)" method on the mock object.
    verify(emailService).sendMail("sj@xyz.com");

  /*
        Without timeout(500), this test would FAIL because:
      - placeOrder() returns at ~2ms
       - verify() runs at ~3ms
       - But email is sent at ~100ms (in background thread)

         With timeout(500):
       - Mockito polls every few ms: "Was sendMail called?"
        - At ~100ms, yes Test passes.

    timeout returns as soon as invocation encounter (say at 150ms,
    so it will not wait till 500ms). But max it will wait for 500ms
  */
  verify(emailService, timeout(500)).sendMail("sj@xyz.com");
}

```

But sometime, there are scenarios, where we want to wait for full time duration, in that case use ***after(ms)***

```
@Test
void testAsyncOrderService_WithAfter() {
    // Arrange
    OrderRepository orderRepo = mock(OrderRepository.class);
    NotificationService emailService =
mock(NotificationService.class);
    AsyncOrderService service = new AsyncOrderService(orderRepo,
emailService);

    // Act
    service.placeOrder("sj@xyz.com");

    verify(orderRepo).save();
```

```
  verify(emailService, after(500).times(1)).sendMail("sj@xyz.com");
}

```

Advantage of after(ms) is:

- *First it will wait for full 500ms. Not like timeout(ms) where it return as soon as first invocation of sendMail encounters.*
- *So after(ms) has complete info, how many times "sendMail"* is invoked. So I did after(500).times(1), means sendMail should be invoked only 1 time.